

TEST REPORT

Replicated Counter Service: gRPC, Lamport Clocks and Majority Quorum

Project: MirasTask_1

Test execution: 21 automated tests passed in 122.84 seconds

Environment: Windows / Python 3.10.9 / pytest 9.1.1

This report documents the implementation and verification evidence present in the submitted project archive. Quantitative results are taken from logs/perf_results.md and the supplied pytest console output. No unrecorded measurements are presented as observed facts.

1. Scope and requirements under test

The system exposes Increment and Get RPCs for named counters. Clients use deadlines and bounded retries; an idempotency key is reused across attempts so a timed-out operation is not applied twice. Each process maintains a Lamport logical clock. The quorum client sends writes to three independent replicas and reports a write as committed only after at least two acknowledgements.

Assignment area	Implementation / verification evidence	Result
A2 - RPC failure handling	Deadline and bounded retry behavior in client.py; same idempotency key on retries; duplicate suppression checked by test_duplicate_key_not_reapplied and test_retry_after_timeout_is_safe.	PASS
A3 - concurrent access	Shared state and deduplication guarded by a lock; test_concurrent_increments_exact checks two concurrent workers produce exactly 2,000 increments.	PASS
B1 - Lamport clocks	Thread-safe LamportClock tick/receive rules; tests cover increment, receive-max-plus-one, monotonicity under threads, and log format.	PASS
C1 - replication/quorum	Quorum commit after 2 of 3 acknowledgements; tests cover one replica down, below-majority failure, convergence, and partial application.	PASS
C2 - unit/integration	Counter mutation, missing counter, deduplication, concurrency, retry, quorum, convergence, and edge cases.	PASS
C3 - failure injection	Process-level crash, duplicate request, induced timeout/retry, and two-replica failure tests; logs saved under logs/failures/.	PASS
C4 - performance	Three runs per configuration, at least 2,000 requests per run; median and nearest-rank p95 reported.	RECORDED

2. Test environment and method

Operating system and runtime: Windows (the benchmark record specifies Windows 10), Python 3.10.9, 12 logical CPUs. The gRPC server uses a ThreadPoolExecutor with max_workers=8. Client and replicas run on the same machine over 127.0.0.1; therefore these measurements describe local-process/network behavior, not a geographically distributed deployment.

Recorded package versions: grpcio 1.84.0; grpcio-tools 1.84.0; protobuf 7.36.2; pytest 9.1.1; pluggy 1.6.0. The complete package list is included in Appendix A.

Automated tests were executed with: `python -m pytest tests -v`. The supplied console output reports 21 tests collected and 21 passed in 122.84 seconds. The benchmark script warms up each client, measures call latency with `time.perf_counter()`, pools three runs, and computes p95 using the nearest-rank index $\text{ceil}(0.95 \times n) - 1$.

3. Test cases and observed results

The actual-result column summarizes the assertion exercised by the test and the supplied all-green test run. The console output gives aggregate pass/fail status; detailed numeric outcomes below are those explicitly asserted in the test source.

ID	Purpose	Expected vs actual result	Status
T01	Lamport tick increments	Clock increases by exactly one	PASS
T02	Receive when remote clock is ahead	Local clock becomes max(local, remote)+1	PASS
T03	Receive when remote clock is behind	Local clock still increments by one	PASS
T04	Clock under threads	Clock never moves backwards	PASS
T05	Log formatting	Event lines match required format	PASS
T06	Increment delta	Increment applies requested delta	PASS
T07	Duplicate idempotency key	Same key returns stored value; mutation occurs once	PASS
T08	Concurrent increments	Two clients × 1,000 increments yield exactly 2,000	PASS
T09	Get missing counter	found=False and value=0	PASS
T10	Timeout then retry	Counter changes exactly once with reused key	PASS
T11	Quorum with one replica stopped	2/3 acknowledgements; committed=True	PASS
T12	Below-majority write	1/3 acknowledgement; committed=False	PASS
T13	Replica convergence	All three replicas reach equal values after batch writes	PASS
T14	Counter ID isolation	Independent counters retain separate values	PASS
T15	Same key through quorum	Duplicate recognized by all replicas; value remains 5	PASS
T16	Failed write partial application	Write reported uncommitted, surviving replica may contain value 1	PASS

T17	Negative delta	Increment by -4 after value 10 yields 6	PASS
T18	Replica crash mid-request	Remaining two replicas acknowledge; caller receives committed result	PASS
T19	Request duplication	Repeated key applied once on all replicas	PASS
T20	Induced timeout with retry	Retry succeeds; all replicas converge to value 1	PASS
T21	Two crashes below majority	Only one acknowledgement; write not reported committed	PASS

Interpretation: the suite checks basic increments as well as concurrent exactness, retry safety, duplicate suppression, quorum threshold behavior, and replica convergence. `test_failed_write_may_partially_apply` intentionally documents that a write can be applied on one surviving replica even though the client refuses to label it committed because a majority was not reached.

4. Failure-injection results

Failure tests launch replicas as separate operating-system processes and kill or delay them to make failure behavior externally observable. Scenario-specific client and replica logs are stored under logs/failures/.

Scenario	Injected fault	Invariant / observed result	Evidence
Replica crash mid-request	Replica B delayed, then killed while a write is in flight.	Write still commits with two acknowledgements; no exception reaches the caller; replicas A and C contain value 1.	test_replica_crash_mid_request; logs/failures/crash_mid_request/
Request duplication	Same counter increment and idempotency key sent twice.	Second operation is reported duplicate; value remains 1 on all three replicas.	test_request_duplication; logs/failures/request_duplication/
Timeout with retry	Replica B delays first request by 1.5 s while client deadline is 0.5 s.	Retry succeeds; 3 acknowledgements; all replicas contain value 1.	test_induced_timeout_with_retry; logs/failures/induced_timeout_retry/
Two replicas unavailable	Replicas B and C killed.	Only one acknowledgement remains; client reports committed=False.	test_two_crashes_below_majority; logs/failures/below_majority/

Quorum interpretation and anomaly

With three replicas, two acknowledgements are required. One unavailable replica can be tolerated because the other two may acknowledge; with two unavailable replicas, the group cannot form a majority and the client does not report success. Safety is weaker than in a consensus-based replicated log: a below-majority write can still mutate the lone available replica, as test_failed_write_may_partially_apply demonstrates. A protocol such as Raft coordinates a replicated log and commit index so client-visible committed entries follow its quorum rules; this assignment does not implement leader election or log replication.

Consistency risk of Get

Get reads from one replica, so it may return a stale value if that replica has not applied a write already applied elsewhere. Independent in-memory replica state also means restarting a replica loses its local counters and deduplication history.

5. Performance check (Task C4)

The benchmark ran three repetitions per configuration with at least 2,000 measured requests in each repetition. The pooled table contains 6,000 measured requests per configuration.

Configuration	Median latency (ms)	p95 latency (ms)	Requests
Single replica, 1 client	0.33	0.36	6,000
Single replica, 16 clients	2.99	3.15	6,000
Quorum (3 replicas), 1 client	0.59	0.62	6,000
Quorum (3 replicas), 16 clients	7.42	10.49	6,000

Additional recorded throughput (mean $\pm$ standard deviation over three runs): single replica / 1 client $3,010 \pm 12$ requests/s; single replica / 16 clients $5,336 \pm 15$ requests/s; quorum / 1 client $1,695 \pm 6$ requests/s; quorum / 16 clients $2,164 \pm 160$ requests/s.

Interpretation

At one client, quorum median latency is 0.59 ms versus 0.33 ms for a single replica, reflecting fan-out to three endpoints and waiting for acknowledgements. At 16 clients, median latency rises to 7.42 ms on the quorum path versus 2.99 ms for one replica, and quorum p95 reaches 10.49 ms. The additional work and coordination cost are visible, especially under concurrency, while the service gains tolerance of one replica failure for majority writes. For a backend where availability is more important than minimum latency, this overhead may be acceptable; the decision should be validated against the application's latency target. These are loopback measurements on one Windows machine and should not be generalized to a multi-host network.

6. Lamport-clock ordering analysis (Task B2)

The captured scenario logs show client-1 incrementing x twice, client-2 incrementing y twice, and client-3 issuing Get calls. The client logs provide these local sequences:

```
[client-1] SEND Increment(counter=x, delta=1) L=1
[client-1] RECV IncrementReply(new_value=1) L=10 (received L=9)
[client-1] SEND Increment(counter=x, delta=1) L=11
[client-1] RECV IncrementReply(new_value=2) L=16 (received L=15)

[client-2] SEND Increment(counter=y, delta=1) L=1
[client-2] RECV IncrementReply(new_value=1) L=8 (received L=7)
```

```
[client-2] SEND Increment(counter=y, delta=1) L=9
```

```
[client-2] RECV IncrementReply(new_value=2) L=14 (received L=13)
```

Causally ordered pairs

1) client-1 SEND Increment(x) at L=1 happens before its corresponding reply receive at L=10: the request reaches a replica, is processed, and the reply returns to the client. 2) client-1's first reply receive at L=10 happens before its second send at L=11 by local program order. The second chain illustrates the happened-before relation.

Concurrent events and clock limitation

The first sends from client-1 and client-2 both carry L=1, but belong to independent processes and neither client sends a message to the other. They are concurrent in the captured trace; equal or different Lamport values alone do not establish causal order. Lamport clocks preserve $a \rightarrow b$ implies $L(a) < L(b)$, but the converse is false. Vector clocks can distinguish more concurrent event histories because they track per-process causal knowledge.

7. Reflection and known limitations

Why a timeout is ambiguous

After a timeout, the client cannot know whether the request was lost, the server executed it but the reply was lost, or the server is merely slow. The design makes retries safe by generating one UUID idempotency key per logical operation, reusing it for every retry, and checking/storing the key in the same critical section as the counter mutation. This provides at-most-once mutation per key while the replica's in-memory deduplication state remains available.

Safety and liveness under failures

With three replicas, the service can return a successful quorum result when one replica is down, because two acknowledgements remain possible. With two replicas down, it cannot reach a majority and must report the write as uncommitted. The implementation demonstrates a partial-application anomaly for such failed writes, so its simple quorum acknowledgement rule is not a full consensus protocol and does not provide all safety guarantees of Raft.

Smallest change for restart survival

Persisting counter state and the deduplication store to durable storage before acknowledging a mutation would be a minimal improvement for survival across ordinary process restarts. This anticipates replicated-log and durable-state mechanisms studied in consensus protocols, but persistence alone would not make the current independent-replica design equivalent to Raft.

Known limitations

- Counter values and idempotency keys are in memory; a restarted replica loses both.
- Writes are sent independently to replicas; there is no leader election, ordered replicated log, or consensus protocol. A write that fails to reach a majority can still partially apply.
- Get reads from a single replica and can be stale.
- Performance measurements use loopback networking on one Windows machine; latency and throughput will differ on real networks and under production load.

Conclusion

The provided execution demonstrates a green 21-test suite across clock rules, counter correctness, concurrency, retries, quorum semantics, and injected failures. The benchmark satisfies the four required configurations and reports median and p95 latency. Principal limitations are durability across restart, read consistency, and absence of consensus-level coordination.

Appendix A. Environment package list

Captured from logs/pip_freeze.txt in the submitted archive:

```
colorama==0.4.6  
  
exceptiongroup==1.3.1  
  
grpcio==1.84.0  
  
grpcio-tools==1.84.0  
  
iniconfig==2.3.1  
  
packaging==26.3  
  
pluggy==1.6.0  
  
protobuf==7.36.2  
  
Pygments==2.21.0  
  
pytest==9.1.1  
  
tomli==2.5.0  
  
typing_extensions==4.16.0
```

Appendix B. Captured event-log excerpts

The repository contains separate files logs/client-1.log, logs/client-2.log, logs/client-3.log, and replica/failure logs. Representative client events are reproduced below; interleaving may differ between runs.

```
[client-1] SEND Increment(counter=x, delta=1) L=1  
  
[client-1] RECV IncrementReply(new_value=1) L=10 (received L=9)  
  
[client-1] SEND Increment(counter=x, delta=1) L=11  
  
[client-1] RECV IncrementReply(new_value=2) L=16 (received L=15)
```

[client-2] SEND Increment(counter=y, delta=1) L=1

[client-2] RECV IncrementReply(new_value=1) L=8 (received L=7)

[client-2] SEND Increment(counter=y, delta=1) L=9

[client-2] RECV IncrementReply(new_value=2) L=14 (received L=13)

[client-3] SEND Get(counter=x) L=1

[client-3] RECV GetReply(value=0, found=False) L=5 (received L=4)

[client-3] SEND Get(counter=y) L=6

[client-3] RECV GetReply(value=2, found=True) L=18 (received L=17)

[client-3] SEND Get(counter=x) L=19

[client-3] RECV GetReply(value=2, found=True) L=22 (received L=21)

[client-3] SEND Get(counter=y) L=23

[client-3] RECV GetReply(value=2, found=True) L=26 (received L=25)

Appendix C. Reproduction commands

```
python -m pytest tests -v
python tests/run_scenario_b2.py
python tests/perf_benchmark.py
pip freeze > logs/pip_freeze.txt
```

Source evidence in the submitted archive: tests/test_clocks.py, tests/test_counter.py, tests/test_failures.py, tests/perf_benchmark.py, logs/perf_results.md, logs/pip_freeze.txt, logs/client-*.log, and logs/failures/*.

